

Spring Container & Bean Lifecycle

Deep Comprehensive Tutorial

Java Backend Series -- Spring Core Internals

A. BeanFactory vs ApplicationContext	Core container interfaces
B. ApplicationContext Startup Sequence	refresh() internals
C. Bean Scopes	Singleton, Prototype, Request, Session
D. Full Bean Lifecycle	12 phases from birth to death
E. @PostConstruct / @PreDestroy	Lifecycle annotations
F. BeanPostProcessor	Hook into bean creation
G. BeanFactoryPostProcessor	Modify bean definitions
H. @Lazy	Deferred bean creation
I. ApplicationContext Events	Event-driven container

A BeanFactory vs ApplicationContext

Core container interfaces -- differences, when each is used

1. Plain English Explanation

BeanFactory is the root interface of the Spring IoC container. It provides the absolute minimum: the ability to hold bean definitions and retrieve beans on demand. Think of it as a blueprint factory -- it knows how to build objects but does nothing until you ask. ApplicationContext extends BeanFactory and adds enterprise-grade features: event publishing, internationalization (MessageSource), automatic BeanPostProcessor registration, AOP auto-proxying, and resource loading. In practice, you almost always use ApplicationContext.

2. Real-World Analogy

BeanFactory is like a basic tool shed -- it stores tools and hands them to you when you ask. ApplicationContext is like a full workshop: same tools, but also an intercom (events), instruction manuals in multiple languages (i18n), automatic safety checks on every tool before you use it (BeanPostProcessors), and a climate control system that pre-warms everything (eager singleton creation).

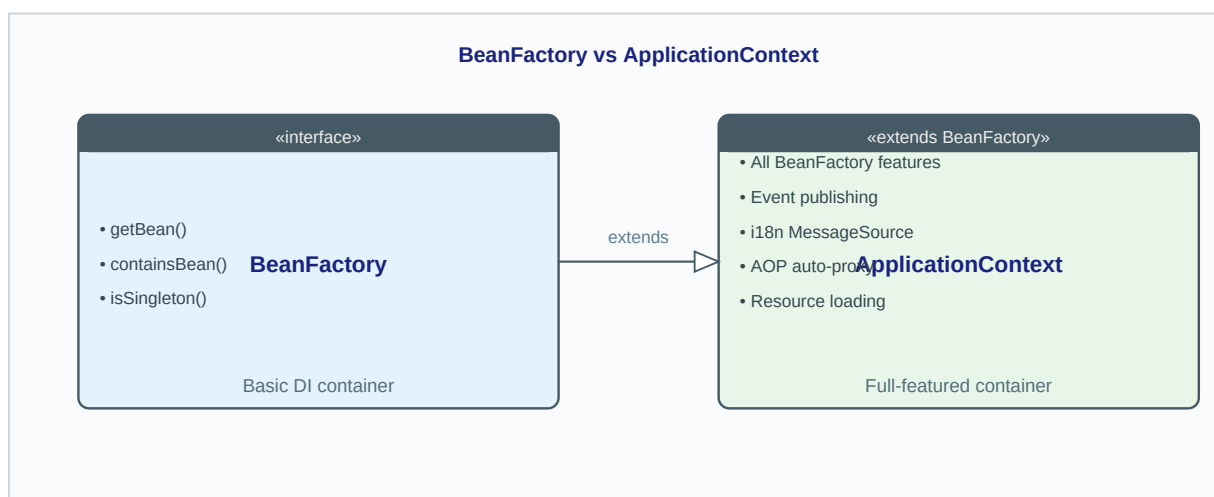
3. Minimal Working Code Example

```
// BeanFactory -- low-level, rarely used directly
DefaultListableBeanFactory factory = new DefaultListableBeanFactory();
XmlBeanDefinitionReader reader = new XmlBeanDefinitionReader(factory);
reader.loadBeanDefinitions("classpath:beans.xml");
MyService svc = factory.getBean(MyService.class);

// ApplicationContext -- what you use in real apps
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
MyService svc2 = ctx.getBean(MyService.class);

// Or with XML
ApplicationContext xmlCtx =
    new ClassPathXmlApplicationContext("classpath:beans.xml");
```

4. Diagram



5. Behind the Scenes -- Spring Internals

BeanFactory is defined in `org.springframework.beans.factory.BeanFactory`. The main production implementation is `DefaultListableBeanFactory`, which also implements `BeanDefinitionRegistry` (stores `BeanDefinition` objects) and `HierarchicalBeanFactory` (parent-child contexts). `ApplicationContext` implementations (`ClassPathXmlApplicationContext`, `AnnotationConfigApplicationContext`, `GenericWebApplicationContext`) all delegate to an internal `DefaultListableBeanFactory` instance. The critical difference: `ApplicationContext` automatically detects and registers `BeanPostProcessor` and `BeanFactoryPostProcessor` beans during `refresh()`, whereas with a raw `BeanFactory` you must register them manually. `ApplicationContext` also pre-instantiates all non-lazy singletons at startup via `finishBeanFactoryInitialization()` then `preInstantiateSingletons()`, which catches wiring errors early rather than at first use.

6. Realistic Practical Example

```
// Production application -- always use ApplicationContext
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        // SpringApplication creates AnnotationConfigServletWebServerAppCtx
        SpringApplication.run(MyApp.class, args);
    }
}

// Accessing context programmatically
@Component
public class MyService implements ApplicationContextAware {
    private ApplicationContext ctx;

    @Override
    public void setApplicationContext(ApplicationContext ctx) {
        this.ctx = ctx;
    }

    public void doWork() {
        // Access sibling beans, publish events, load resources
        ctx.publishEvent(new MyCustomEvent(this));
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: BeanFactory is deprecated

Reality: No -- it is still the root interface. `DefaultListableBeanFactory` is used internally by every `ApplicationContext`.

Mistake: ApplicationContext is just a thin wrapper

Reality: Wrong. It adds ~20 additional features including auto-BPP detection, event multicasting, and eager singleton creation.

Mistake: BeanFactory is faster for production

Reality: Only in the narrowest micro-containers. In server-side Java, `ApplicationContext` startup cost is paid once.

Mistake: You need `ClassPathXmlApplicationContext`

Reality: Since Spring 3, `AnnotationConfigApplicationContext` with `@Configuration` classes is preferred over XML.

B ApplicationContext Startup Sequence

The exact step-by-step startup -- refresh() internals

1. Plain English Explanation

When you create an `ApplicationContext`, it runs a method called `refresh()`. This single method orchestrates everything: loading bean definitions, running factory post-processors, registering post-processors, and finally creating all eager singleton beans. Understanding `refresh()` means you understand when your code runs and why certain things must happen in a specific order.

2. Real-World Analogy

Think of `refresh()` as opening a restaurant for the day. First you read the menu (load bean definitions). Then the health inspector visits (`BeanFactoryPostProcessors` can modify definitions). Then you assign staff roles (register `BeanPostProcessors`). Finally you prep all the standard dishes before the first customer (eager singleton instantiation). Only after all this is done do you open the doors.

3. Minimal Working Code Example

```
// The entire startup is triggered by this one line
ApplicationContext ctx =
    new AnnotationConfigApplicationContext(AppConfig.class);
// By this line, ALL singletons are created and wired.

// You can also trigger refresh manually on a GenericApplicationContext:
GenericApplicationContext gac = new GenericApplicationContext();
gac.registerBeanDefinition("myBean",
    BeanDefinitionBuilder.rootBeanDefinition(MyService.class)
        .getBeanDefinition());
gac.refresh(); // Now beans are created
MyService svc = gac.getBean(MyService.class);
```

4. Diagram -- Startup Sequence

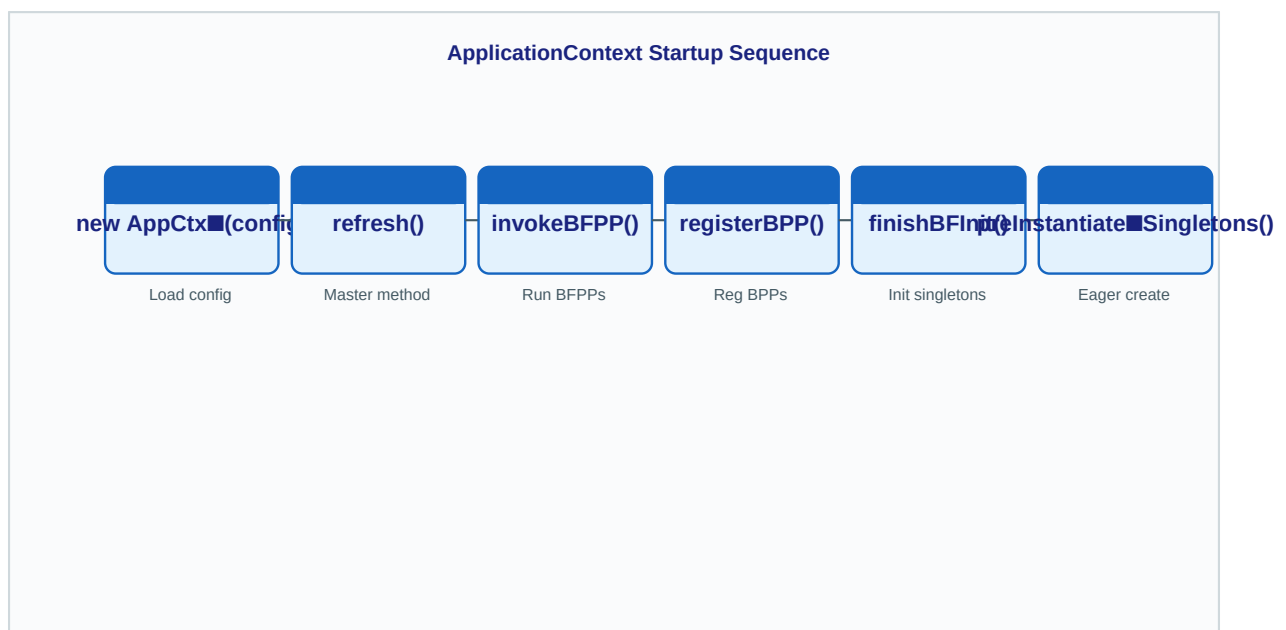


Fig B-1: Six key phases inside `AbstractApplicationContext.refresh()`

5. Behind the Scenes -- Spring Internals

All `ApplicationContext` implementations extend `AbstractApplicationContext`, which defines the `refresh()` template method (Template Method pattern). The full call chain:

```
AbstractApplicationContext.refresh() {
    prepareRefresh();                // set start time, early events
    ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
    // For XML: loads XML, parses BeanDefinitions
    // For annotations: ClassPathBeanDefinitionScanner scans @Component
    prepareBeanFactory(beanFactory); // register env beans, SpEL resolver
    postProcessBeanFactory(beanFactory); // hook for subclasses (e.g. web)
    invokeBeanFactoryPostProcessors(beanFactory);
    // Run ConfigurationClassPostProcessor (processes @Configuration)
    // Then all other BeanFactoryPostProcessors in order
    registerBeanPostProcessors(beanFactory);
    // Find all BPP beans, sort by PriorityOrdered/Ordered/rest
    // Register but do NOT call yet
    initMessageSource();             // i18n
    initApplicationEventMulticaster(); // event bus
    onRefresh();                     // web: create servlet container
    registerListeners();
    finishBeanFactoryInitialization(beanFactory);
    // beanFactory.preInstantiateSingletons()
    // For each non-lazy singleton: getBean() -> doCreateBean()
    finishRefresh();                 // publish ContextRefreshedEvent
}
```

The key class is `AbstractApplicationContext` (spring-context module). `ConfigurationClassPostProcessor` (a `BeanDefinitionRegistryPostProcessor`) runs first in `invokeBeanFactoryPostProcessors` and processes `@Configuration`, `@ComponentScan`, `@Bean`, `@Import` annotations -- generating synthetic `BeanDefinitions` for `@Bean` methods.

6. Realistic Practical Example

```
// Custom hook that runs AFTER all beans are created
@Component
public class StartupValidator
    implements ApplicationListener<ContextRefreshedEvent> {
    @Autowired
    private List<MyService> services;

    @Override
    public void onApplicationEvent(ContextRefreshedEvent event) {
        // All singletons ready -- safe to validate cross-bean state
        services.forEach(MyService::validate);
        System.out.println("All " + services.size() + " services validated.");
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: Calling `refresh()` twice is safe

Reality: For most contexts it is not -- calling `refresh()` on an already-active context throws `IllegalStateException`.

Mistake: Bean definitions are loaded lazily on first `getBean()`

Reality: No. All bean definitions are loaded during `obtainFreshBeanFactory()` at startup. Only bean instances may be lazy.

Mistake: `BeanPostProcessors` run during `invokeBeanFactoryPostProcessors`

Reality: No -- they are only registered there. They execute later, during `doCreateBean()`, for each individual bean.

C

Bean Scopes

Singleton, Prototype, Request, Session -- with diagrams

1. Plain English Explanation

A bean's scope defines how many instances are created and how long each lives. Singleton (default): one shared instance per ApplicationContext, created at startup. Prototype: a brand-new instance is created every time the bean is requested. Request: one instance per HTTP request (web apps only). Session: one instance per HTTP session (web apps only). Spring 5 also has application scope (one per ServletContext) and websocket scope.

2. Real-World Analogy

Singleton is like the one shared printer in an office -- everyone uses the same machine. Prototype is like a disposable coffee cup -- each person gets a new one. Request scope is like a form you fill out for a single transaction -- discarded after. Session scope is like your shopping cart -- persists during your visit, gone when you leave.

3. Minimal Working Code Example

```
@Component
// @Scope("singleton") // default -- no annotation needed
public class SharedCache { ... }

@Component
@Scope("prototype")
public class ReportBuilder { ... } // new instance each getBean()

@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST,
        proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext { ... } // new per HTTP request

@Component
@Scope(value = WebApplicationContext.SCOPE_SESSION,
        proxyMode = ScopedProxyMode.TARGET_CLASS)
public class UserCart { ... } // per session

// Injecting a prototype into a singleton -- fix with ObjectFactory:
@Component
public class OrderService {
    @Autowired
    private ObjectFactory<ReportBuilder> builderFactory;

    public void process() {
        ReportBuilder b = builderFactory.getObject(); // new each time
    }
}
```

4. Diagram -- Bean Scopes

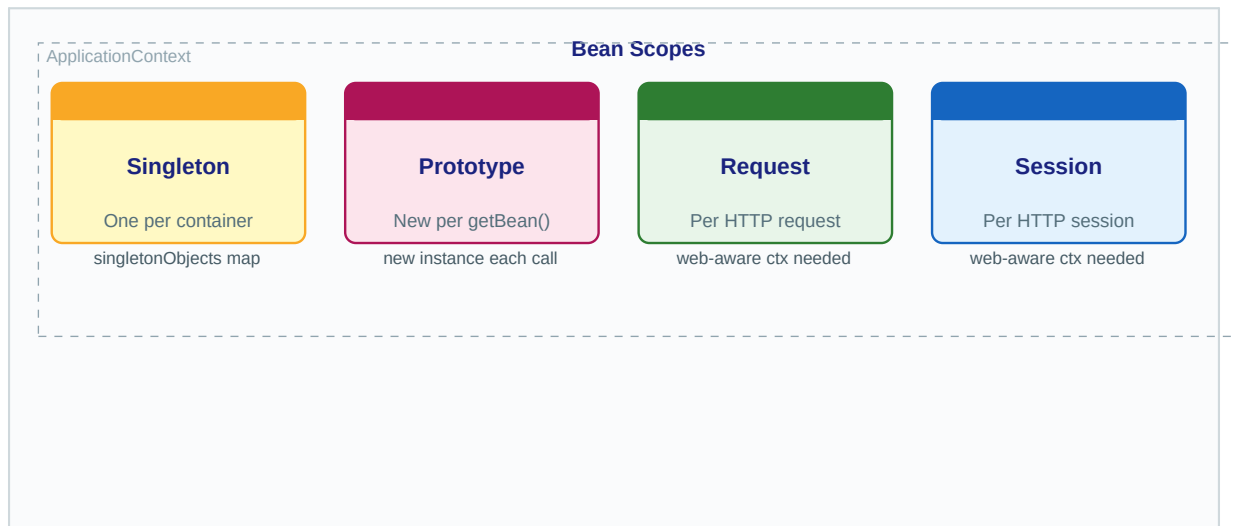


Fig C-1: The four primary bean scopes and their lifecycle boundaries

5. Behind the Scenes -- Spring Internals

Singleton scope is managed by `DefaultSingletonBeanRegistry`. It maintains three maps (the three-level cache) to handle circular dependencies:

```
// DefaultSingletonBeanRegistry internals
Map<String, Object> singletonObjects;           // L1: fully initialized beans
Map<String, Object> earlySingletonObjects;      // L2: early exposed, not yet init
Map<String, ObjectFactory<?>> singletonFactories; // L3: factory for early ref

// Prototype scope: no caching at all
// AbstractBeanFactory.doGetBean() for prototype:
if (mbd.isPrototype()) {
    Object prototypeInstance = null;
    beforePrototypeCreation(beanName);
    prototypeInstance = createBean(beanName, mbd, args);
    afterPrototypeCreation(beanName);
    return prototypeInstance; // new object, Spring forgets it
}
```

For request/session scopes, Spring uses a `ScopedProxyFactoryBean` that wraps the real bean in a CGLIB proxy. The proxy is injected into singletons, but each method call on the proxy is delegated to the real bean retrieved from the current request/session attributes via `RequestContextHolder`.

6. Realistic Practical Example

```
// Session-scoped shopping cart injected into singleton controller
@RestController
@RequestMapping("/cart")
public class CartController {
    // Works because UserCart is a scoped proxy
    @Autowired
    private UserCart userCart; // proxy injected once at startup

    @PostMapping("/add")
    public ResponseEntity<Void> addItem(@RequestBody Item item) {
        userCart.add(item); // proxy delegates to session-specific instance
        return ResponseEntity.ok().build();
    }
}
```

```
@Component
@Scope(value = "session", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class UserCart implements Serializable {
    private List<Item> items = new ArrayList<>();
    public void add(Item i) { items.add(i); }
    public List<Item> getItems() { return items; }
}
```

7. Common Mistakes and Misconceptions

Mistake: Singleton beans are thread-safe

Reality: No! Singleton means one shared instance. If it has mutable state, you need explicit synchronization. Prefer stateless singletons.

Mistake: Prototype beans have their destroy methods called

Reality: Spring does NOT call @PreDestroy or destroy-method on prototype beans. Spring hands them off and forgets them.

Mistake: Injecting a short-lived bean into a singleton just works

Reality: Wrong -- injecting a prototype or request-scoped bean into a singleton gives you one instance forever. Use ObjectFactory or proxyMode.

Mistake: @Scope without proxyMode works for web scopes

Reality: Without proxyMode = TARGET_CLASS, injecting a request/session bean into a singleton will fail or give wrong-scope behavior.

D Full Bean Lifecycle

12 phases from instantiation to destruction

1. Plain English Explanation

Every Spring bean goes through a well-defined lifecycle from the moment its definition is registered until the container shuts down. The lifecycle has 12 distinct phases: definition loading, instantiation, property injection, three Aware callbacks, BeanPostProcessor before-init, @PostConstruct/afterPropertiesSet, init-method, BeanPostProcessor after-init, the in-use period, and finally @PreDestroy/destroy. Understanding this order is essential for writing correct initialization and cleanup code.

2. Real-World Analogy

Think of a new employee's journey: (1) HR creates a job description (definition), (2) they are hired (instantiated), (3) given a desk and equipment (properties), (4) told their employee ID (BeanNameAware), (5) told who their manager is (BeanFactoryAware), (6) given a company ID badge (ApplicationContextAware), (7) an HR pre-check before onboarding tasks (BPP before), (8) orientation day (@PostConstruct), (9) department-specific training (init-method), (10) HR post-check (BPP after), (11) doing their job (in use), (12) exit interview and badge return (@PreDestroy).

3. Minimal Working Code Example

```
@Component
public class FullLifecycleBean
    implements BeanNameAware, BeanFactoryAware,
               ApplicationContextAware,
               InitializingBean, DisposableBean {

    public FullLifecycleBean() {
        System.out.println("2. Instantiated");
    }

    @Value("${app.name}")
    private String appName; // Phase 3: property injection

    @Override
    public void setBeanName(String name) { // Phase 4
        System.out.println("4. BeanNameAware: " + name);
    }

    @Override
    public void setBeanFactory(BeansFactory bf) { // Phase 5
        System.out.println("5. BeanFactoryAware");
    }

    @Override
    public void setApplicationContext(ApplicationContext ctx) { // Phase 6
        System.out.println("6. ApplicationContextAware");
    }

    @PostConstruct
    public void postConstruct() { // Phase 8
        System.out.println("8. @PostConstruct");
    }
}
```

```

@Override
public void afterPropertiesSet() { // also Phase 8
    System.out.println("8b. afterPropertiesSet");
}

@PreDestroy
public void preDestroy() { // Phase 12
    System.out.println("12. @PreDestroy");
}

@Override
public void destroy() { // also Phase 12
    System.out.println("12b. DisposableBean.destroy");
}
}

```

4. Diagram -- Full Bean Lifecycle

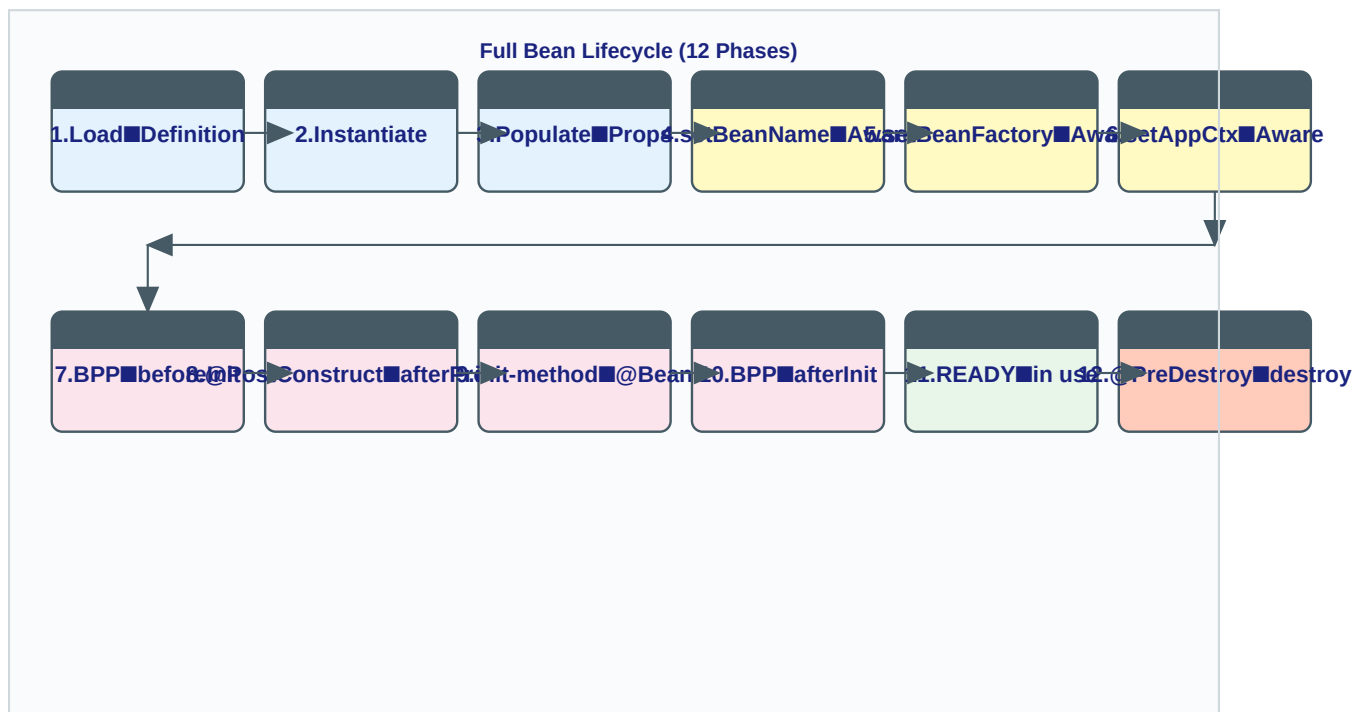


Fig D-1: All 12 lifecycle phases; top row=setup, bottom row=init+use+destroy

5. Behind the Scenes -- Spring Internals

The lifecycle is implemented in `AbstractAutowireCapableBeanFactory.doCreateBean()`:

```

doCreateBean(beanName, mbd, args) {
    // Phase 2: instantiation
    BeanWrapper bw = createBeanInstance(beanName, mbd, args);

    // Phase 3: populate properties
    populateBean(beanName, mbd, bw);
    // @Autowired resolved by AutowiredAnnotationBeanPostProcessor

    // Phases 4-10: initialization
    initializeBean(beanName, bean, mbd) {
        invokeAwareMethods(beanName, bean);
        // BeanNameAware.setBeanName() (Phase 4)
    }
}

```

```

        // BeanClassLoaderAware                (Phase 4b)
        // BeanFactoryAware.setBeanFactory()    (Phase 5)

        wrappedBean = applyBPPsBeforeInitialization(bean);
        // ApplicationContextAwareProcessor -> setAppCtx (Phase 6)
        // CommonAnnotationBPP -> @PostConstruct (Phase 8)

        invokeInitMethods(beanName, wrappedBean, mbd);
        // afterPropertiesSet() if InitializingBean (Phase 8b)
        // init-method / @Bean(initMethod)      (Phase 9)

        wrappedBean = applyBPPsAfterInitialization(bean);
        // AbstractAutoProxyCreator creates AOP proxy (Phase 10)
    }
}

```

6. Realistic Practical Example

```

@Component
public class DatabaseConnectionPool {
    @Autowired
    private DataSource dataSource;

    @Value("${db.pool.size:10}")
    private int poolSize;

    private List<Connection> pool;

    @PostConstruct // Phase 8: run after all properties injected
    public void init() throws Exception {
        pool = new ArrayList<>(poolSize);
        for (int i = 0; i < poolSize; i++) {
            pool.add(dataSource.getConnection());
        }
        System.out.println("Pool initialized with " + poolSize + " connections");
    }

    @PreDestroy // Phase 12: called before context closes
    public void cleanup() throws Exception {
        for (Connection c : pool) {
            c.close();
        }
        System.out.println("All connections closed");
    }
}

```

7. Common Mistakes and Misconceptions

Mistake: Using the constructor for initialization that needs injected fields

Reality: Injected fields are NOT set in the constructor. Use `@PostConstruct` for initialization depending on `@Autowired` or `@Value`.

Mistake: `@PostConstruct` and `afterPropertiesSet()` are the same

Reality: Both run in Phase 8, but `@PostConstruct` fires first (processed by BPP), then `afterPropertiesSet()` is called directly.

Mistake: The order is BPP-after then init-method

Reality: Wrong. Order is: BPP-before -> @PostConstruct -> afterPropertiesSet -> init-method -> BPP-after.

Mistake: destroy() and @PreDestroy are called for all beans

Reality: For prototype beans, Spring does NOT call @PreDestroy or destroy(). The garbage collector is responsible.

E **@PostConstruct and @PreDestroy**

When called, which processor handles them

1. Plain English Explanation

@PostConstruct marks a method to be called after the bean is fully constructed and all dependencies are injected, but before the bean is put into service. It is the safest place to run initialization logic that relies on injected dependencies. @PreDestroy marks a method to be called just before the bean is removed from the container -- ideal for releasing resources like database connections, file handles, or scheduled tasks.

2. Real-World Analogy

@PostConstruct is like a surgeon scrubbing in before an operation -- the preparation that happens after they arrive and are fully equipped, before the first incision. @PreDestroy is the post-operation debrief and equipment sterilization -- cleanup that happens before the surgeon leaves.

3. Minimal Working Code Example

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;

@Component
public class CacheManager {
    @Autowired
    private RedisTemplate<String, Object> redis;

    private Map<String, Object> localCache;

    @PostConstruct // safe: redis is already injected
    public void warmUp() {
        localCache = new HashMap<>();
        Set<String> hotKeys = redis.keys("hot:*");
        for (String key : hotKeys) {
            localCache.put(key, redis.opsForValue().get(key));
        }
        System.out.println("Cache warmed: " + localCache.size() + " entries");
    }

    @PreDestroy
    public void flush() {
        localCache.forEach((k, v) -> redis.opsForValue().set(k, v));
        localCache.clear();
        System.out.println("Cache flushed to Redis");
    }
}
```

4. Diagram -- @PostConstruct / @PreDestroy in the Lifecycle

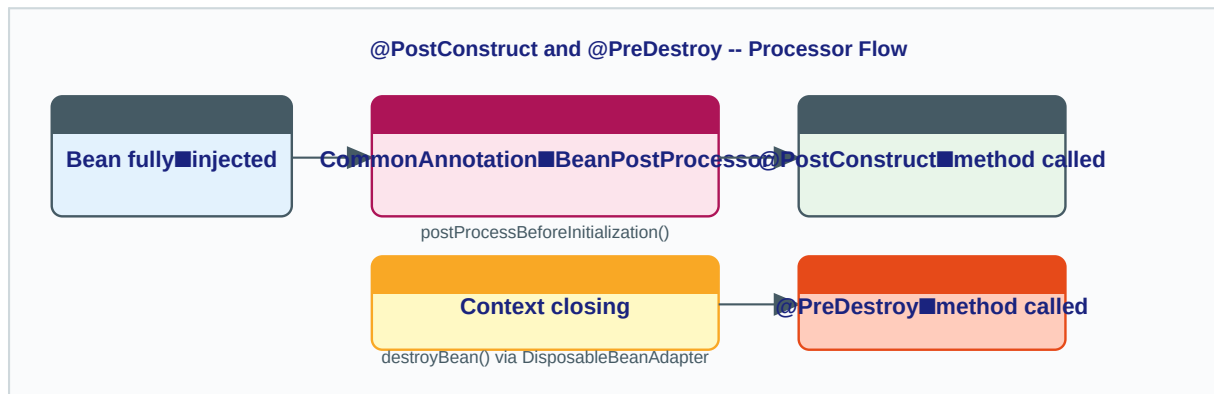


Fig E-1: CommonAnnotationBeanPostProcessor handles both @PostConstruct and @PreDestroy

5. Behind the Scenes -- Spring Internals

Both annotations are defined in Jakarta Annotations (jakarta.annotation). They are processed by CommonAnnotationBeanPostProcessor, registered automatically by AnnotationConfigApplicationContext and . For @PostConstruct: CommonAnnotationBPP implements postProcessBeforeInitialization(). It reflects on the bean class, finds @PostConstruct methods via InitDestroyAnnotationBeanPostProcessor superclass, and invokes them via reflection. For @PreDestroy: the same BPP implements DestructionAwareBeanPostProcessor. At context close, DisposableBeanAdapter.destroy() calls bpp.postProcessBeforeDestruction(bean, beanName) for each registered DestructionAwareBPP. Prototype beans skip this path entirely.

6. Realistic Practical Example

```

@Component
public class ScheduledTaskManager {
    private ScheduledExecutorService executor;

    @PostConstruct
    public void start() {
        executor = Executors.newScheduledThreadPool(4);
        executor.scheduleAtFixedRate(this::doWork, 0, 30, TimeUnit.SECONDS);
        System.out.println("Scheduler started");
    }

    @PreDestroy
    public void stop() {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
                executor.shutdownNow();
            }
        } catch (InterruptedException e) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
        System.out.println("Scheduler stopped cleanly");
    }

    private void doWork() {
        // periodic task
    }
}
  
```


7. Common Mistakes and Misconceptions

Mistake: @PostConstruct can have any signature

Reality: It must be void, take no parameters, and not be static. Spring will throw an exception at startup if violated.

Mistake: @PostConstruct runs before @Autowired injection

Reality: No -- @PostConstruct is explicitly designed to run AFTER all injection is complete.

Mistake: @PreDestroy is called during garbage collection

Reality: No -- it is called when ApplicationContext is explicitly closed. Not called for prototype beans.

Mistake: @Bean(initMethod) is the same as @PostConstruct

Reality: @Bean(initMethod='init') runs in Phase 9, AFTER @PostConstruct (Phase 8). If you have both, @PostConstruct fires first.

F BeanPostProcessor

Hook into bean creation -- before and after init

1. Plain English Explanation

A BeanPostProcessor (BPP) is a Spring extension point that lets you intercept bean creation and modify or wrap every bean in the container. Spring itself uses BPPs internally for @Autowired, @PostConstruct, AOP proxying, and more. You implement two methods: postProcessBeforeInitialization() runs after injection but before init methods; postProcessAfterInitialization() runs after all init methods and is where AOP proxy creation happens.

2. Real-World Analogy

Imagine an assembly line for new company laptops. Before the laptop is handed to the employee (before-init), security installs antivirus and sets disk encryption. After the final setup (after-init), IT wraps it in a tracking case (the AOP proxy). Every single laptop goes through both checkpoints.

3. Minimal Working Code Example

```
@Component
public class LoggingBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName)
        throws BeansException {
        System.out.println("Before init: " + beanName);
        return bean; // return (possibly replaced) bean
    }

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName)
        throws BeansException {
        System.out.println("After init: " + beanName);
        return bean;
    }
}

// Wrapping in a timing proxy (after-init):
@Override
public Object postProcessAfterInitialization(Object bean, String beanName) {
    if (bean instanceof MyService) {
        return Proxy.newProxyInstance(
            bean.getClass().getClassLoader(),
            new Class[]{MyService.class},
            (proxy, method, args) -> {
                long start = System.nanoTime();
                Object result = method.invoke(bean, args);
                System.out.println(method.getName() + " took "
                    + (System.nanoTime() - start) + "ns");
                return result;
            });
    }
    return bean;
}
```

4. Diagram -- BeanPostProcessor Call Order

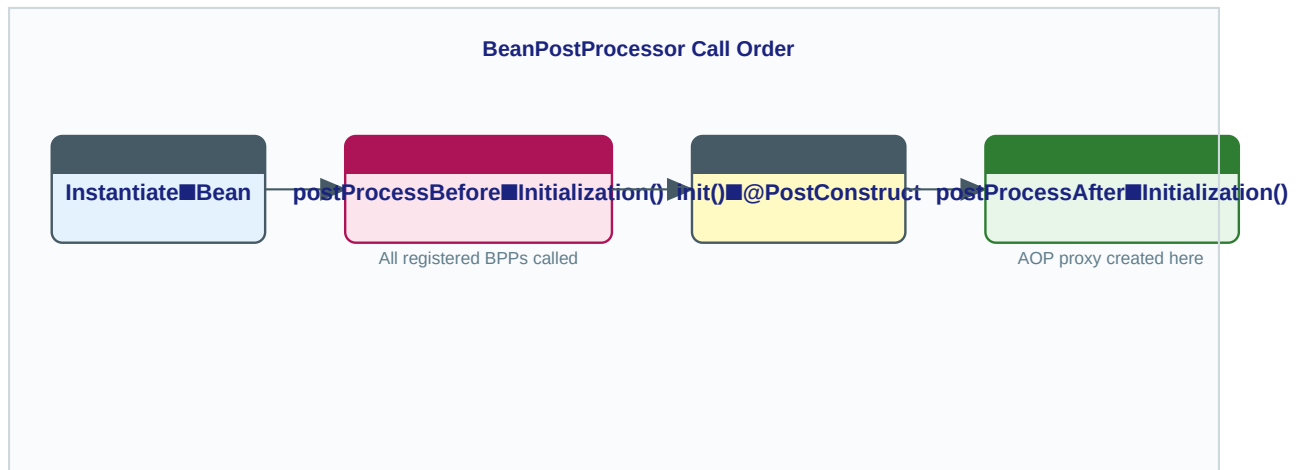


Fig F-1: BPP-before then init then BPP-after; AOP proxy created in postProcessAfterInitialization

5. Behind the Scenes -- Spring Internals

BPPs are registered in `registerBeanPostProcessors()` during `refresh()`. They are sorted: `PriorityOrdered` BPPs first, then `Ordered`, then the rest. Key built-in BPPs:

```
AutowiredAnnotationBeanPostProcessor // @Autowired, @Value, @Inject
CommonAnnotationBeanPostProcessor   // @PostConstruct, @PreDestroy, @Resource
AbstractAutoProxyCreator             // AOP: creates CGLIB/JDK proxies (after-init)
PersistenceAnnotationBeanPostProcessor // @PersistenceContext, @PersistenceUnit
ApplicationContextAwareProcessor     // setApplicationContext() Aware calls

// Call sequence in AbstractAutowireCapableBeanFactory:
wrappedBean = bean;
for (BeanPostProcessor bp : getBeanPostProcessors()) {
    Object current = bp.postProcessBeforeInitialization(wrappedBean, beanName);
    if (current == null) return wrappedBean; // stop on null
    wrappedBean = current;
}
invokeInitMethods(beanName, wrappedBean, mbd);
for (BeanPostProcessor bp : getBeanPostProcessors()) {
    Object current = bp.postProcessAfterInitialization(wrappedBean, beanName);
    if (current == null) return wrappedBean;
    wrappedBean = current;
}
```

Important: BPP beans themselves are NOT processed by other BPPs. Spring intentionally skips BPP post-processing for BPPs to avoid circular startup dependencies. A warning is logged if a bean that depends on a BPP is created before all BPPs are registered.

6. Realistic Practical Example

```
// Audit logging BPP -- wraps every @Service in a proxy
@Component
public class AuditBeanPostProcessor implements BeanPostProcessor {

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) {
        Class<?> cls = AopUtils.getTargetClass(bean);
        if (!cls.isAnnotationPresent(Service.class)) return bean;
    }
}
```

```

        return Proxy.newProxyInstance(
            cls.getClassLoader(),
            cls.getInterfaces(),
            (proxy, method, args) -> {
                String user = SecurityContextHolder.getContext()
                    .getAuthentication().getName();
                System.out.println("[AUDIT] " + user + " called "
                    + cls.getSimpleName() + "." + method.getName());
                return method.invoke(bean, args);
            }
        );
    }
}

```

7. Common Mistakes and Misconceptions

Mistake: Returning null from a BPP method removes the bean

Reality: Returning null stops further BPP processing and uses the bean from the previous step. Never return null -- return the original bean if unmodified.

Mistake: BPPs are called only once per bean class

Reality: They are called for every bean INSTANCE. If you have 10 singleton beans, each BPP is called 10 times.

Mistake: BPP-before and BPP-after always process the same object

Reality: The bean passed to after-init is the RESULT of before-init, which may have been replaced. AOP creates a new proxy in after-init.

Mistake: Order of BPPs does not matter

Reality: Order is critical. AutowiredAnnotationBPP must run before your custom BPP if your BPP inspects injected fields.

G

BeanFactoryPostProcessor

Modify bean definitions before beans are created

1. Plain English Explanation

A BeanFactoryPostProcessor (BFPP) operates on bean DEFINITIONS (the blueprints), not on bean instances. It runs after all bean definitions are loaded but before any singleton beans are instantiated. This gives you a window to change the configuration of any bean -- for example, replacing property placeholders, adding new bean definitions, or changing a bean's class. The most famous BFPP is PropertySourcesPlaceholderConfigurer, which resolves \${...} expressions.

2. Real-World Analogy

Imagine an architect reviews all building blueprints before construction starts. The BFPP is that architect -- they can modify blueprints, add new rooms, or change materials (bean definition properties) before a single brick is laid (before any bean is instantiated). A BPP, by contrast, inspects each finished room after it is built.

3. Minimal Working Code Example

```
@Component
public class MyBeanFactoryPostProcessor implements BeanFactoryPostProcessor {

    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory bf)
        throws BeansException {
        // Inspect all bean definitions
        for (String name : bf.getBeanDefinitionNames()) {
            BeanDefinition bd = bf.getBeanDefinition(name);
            System.out.println("Bean: " + name
                + " class: " + bd.getBeanClassName());
        }

        // Modify a bean definition before instantiation
        BeanDefinition bd = bf.getBeanDefinition("myService");
        bd.getPropertyValues().add("timeout", 5000);

        // Register a new bean definition
        ((BeanDefinitionRegistry) bf).registerBeanDefinition("dynamicBean",
            BeanDefinitionBuilder.rootBeanDefinition(DynamicService.class)
                .addPropertyValue("name", "dynamic")
                .getBeanDefinition());
    }
}

// BeanDefinitionRegistryPostProcessor -- even more powerful:
@Component
public class MyRegistryPP implements BeanDefinitionRegistryPostProcessor {
    @Override
    public void postProcessBeanDefinitionRegistry(
        BeanDefinitionRegistry registry) {
        // Add new bean definitions here -- runs before regular BFPPs
    }
    @Override
    public void postProcessBeanFactory(
        ConfigurableListableBeanFactory bf) { }
```

```
}
```

4. Diagram -- BFPP in the Startup Sequence

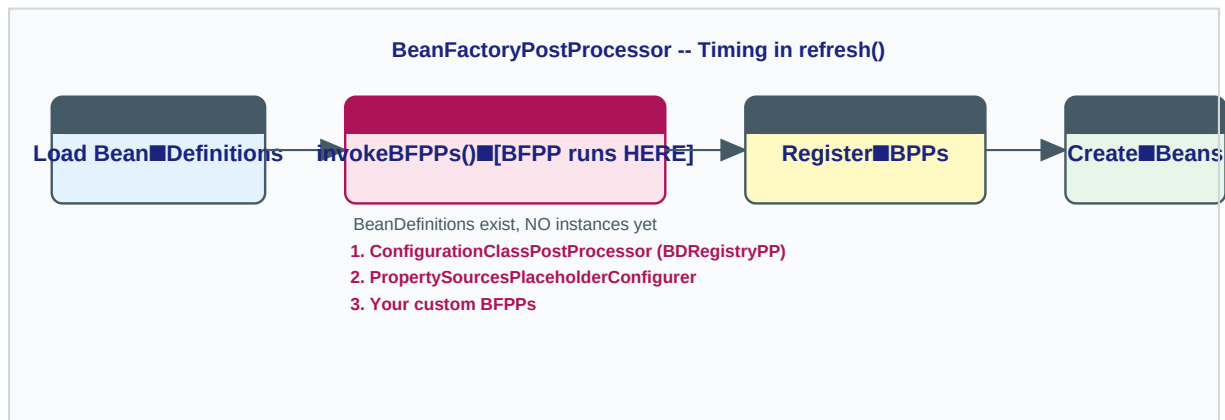


Fig G-1: BFPPs run after definitions are loaded but before any bean is instantiated

5. Behind the Scenes -- Spring Internals

The BFPP interface has one method: `postProcessBeanFactory(ConfigurableListableBeanFactory)`. Its sub-interface `BeanDefinitionRegistryPostProcessor` adds `postProcessBeanDefinitionRegistry(BeansDefinitionRegistry)`, running even earlier. `ConfigurationClassPostProcessor` implements `BeanDefinitionRegistryPostProcessor` and processes all `@Configuration` classes, `@ComponentScan`, `@Bean`, `@Import`, `@ImportResource`. `PropertySourcesPlaceholderConfigurer` scans all `BeanDefinitions` for `{property}` placeholders, resolving them from Environment property sources.

```
// Execution order in PostProcessorRegistrationDelegate.invokeBFPPs():
// 1. BDRegistryPPs implementing PriorityOrdered
//    (ConfigurationClassPostProcessor runs here)
// 2. BDRegistryPPs implementing Ordered
// 3. All remaining BDRegistryPPs
// 4. All regular BFPPs implementing PriorityOrdered
// 5. BFPPs implementing Ordered (PropertySourcesPlaceholderConfigurer)
// 6. All remaining BFPPs
```

6. Realistic Practical Example

```
// Dynamically switch DataSource implementation based on profile
@Component
public class DataSourceOverrideBFPP implements BeanFactoryPostProcessor {

    @Autowired
    private Environment env;

    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory bf) {
        String[] profiles = env.getActiveProfiles();
        if (profiles.length > 0 && "test".equals(profiles[0])) {
            BeanDefinition bd = bf.getBeanDefinition("dataSource");
            bd.setBeanClassName(
                "org.springframework.jdbc.datasource.embedded"
                + ".EmbeddedDatabaseFactoryBean");
            System.out.println("Switched dataSource to H2 for testing");
        }
    }
}
```

```
}  
    }  
}
```

7. Common Mistakes and Misconceptions

Mistake: BFPP and BPP are interchangeable

Reality: No. BFPP modifies bean definitions (blueprints). BPP modifies bean instances (finished objects). They run at completely different times.

Mistake: You can `getBean()` inside a BFPP

Reality: Strongly discouraged. Calling `getBean()` from a BFPP forces premature instantiation, potentially before BPPs are registered.

Mistake: `@Autowired` works reliably in a BFPP

Reality: `@Autowired` fields in a BFPP bean may not be populated because `AutowiredAnnotationBPP` may not be registered yet. Use constructor injection.

1. Plain English Explanation

@Lazy tells Spring: do not create this bean at startup -- create it only when it is first requested. For singleton beans, this trades faster startup time for a slight delay on the first use. When @Lazy is placed at an injection point (on @Autowired), Spring injects a lightweight proxy instead of the real bean -- the proxy creates the real bean on the first method call.

2. Real-World Analogy

Ordering groceries for the whole week on Sunday (eager) vs using a food delivery app that brings groceries only when you decide to cook (lazy). The delivery app (the proxy) is always available, but the actual groceries (real bean) arrive only when you need them. If you never cook that dish, those groceries are never delivered.

3. Minimal Working Code Example

```
// Lazy singleton -- not created until first getBean() call
@Component
@Lazy
public class HeavyReportEngine {
    public HeavyReportEngine() {
        System.out.println("HeavyReportEngine created -- expensive!");
    }
}

// Lazy injection -- proxy injected, real bean deferred
@Component
public class DashboardController {
    @Autowired
    @Lazy
    private HeavyReportEngine reportEngine; // CGLIB proxy at startup

    public void generateReport() {
        reportEngine.generate(); // real bean created HERE on first call
    }
}

// @Lazy on @Configuration applies to all @Bean methods in the class
@Configuration
@Lazy
public class InfraConfig {
    @Bean
    public SlowExternalClient client() {
        return new SlowExternalClient(); // deferred
    }
}
```

4. Diagram -- @Lazy Proxy Mechanism

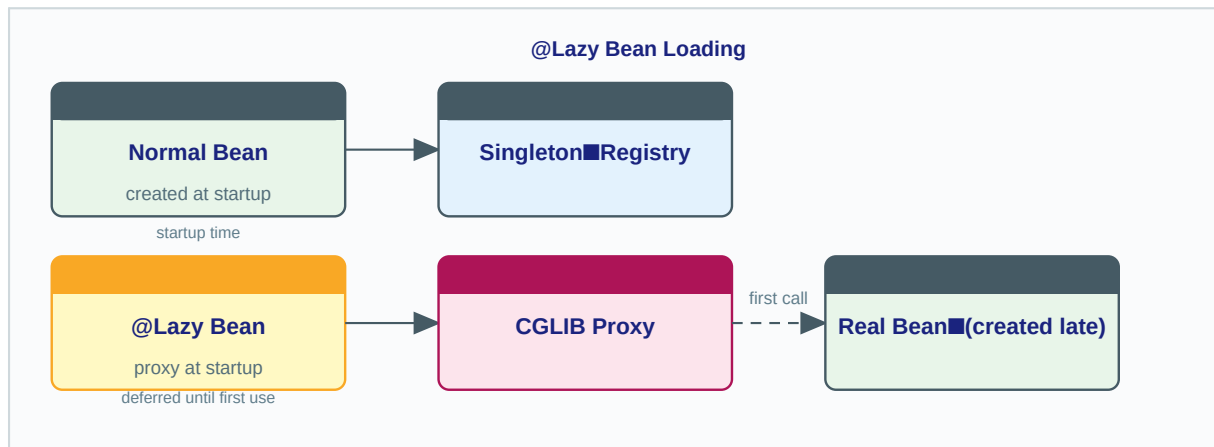


Fig H-1: Normal beans created at startup; @Lazy beans get a CGLIB proxy deferring real creation to first use

5. Behind the Scenes -- Spring Internals

When Spring detects @Lazy on a bean definition, it sets the lazyInit flag in BeanDefinition to true. During preInstantiateSingletons(), this bean is skipped. When @Lazy is placed on an @Autowired injection point, ContextAnnotationAutowireCandidateResolver.buildLazyResolutionProxy() creates a CGLIB subclass proxy:

```
// ContextAnnotationAutowireCandidateResolver
protected Object buildLazyResolutionProxy(
    DependencyDescriptor descriptor, String beanName) {
    BeanFactory beanFactory = getBeanFactory();
    TargetSource ts = new TargetSource() {
        @Override
        public Object getTarget() {
            // First call: creates/retrieves real bean from context
            return beanFactory.getBean(
                beanName, descriptor.getDependencyType());
        }
    };
    ProxyFactory pf = new ProxyFactory();
    pf.setTargetSource(ts);
    pf.setProxyTargetClass(true);
    return pf.getProxy(
        descriptor.getDependencyType().getClassLoader());
}

// preInstantiateSingletons() in DefaultListableBeanFactory:
for (String beanName : beanNames) {
    RootBeanDefinition bd = getMergedLocalBeanDefinition(beanName);
    if (!bd.isAbstract() && bd.isSingleton() && !bd.isLazyInit()) {
        getBean(beanName); // only non-lazy singletons created here
    }
}
}
```

6. Realistic Practical Example

```
@Configuration
public class ExternalServicesConfig {

    @Bean
    @Lazy
    public AnalyticsClient analyticsClient() {
        return new AnalyticsClient("https://analytics.internal:8443");
    }
}
```

```

    }

    @Bean
    @Lazy
    public MLModelService mlModel() {
        return new MLModelService("/models/recommendation-v3.pt");
    }
}

@Service
public class RecommendationService {
    @Autowired(required = false)
    @Lazy
    private MLModelService mlModel;

    public List<String> recommend(String userId) {
        if (mlModel != null) {
            return mlModel.predict(userId);
        }
        return fallbackRecommendations(userId);
    }
}

```

7. Common Mistakes and Misconceptions

Mistake: @Lazy makes prototype beans lazy

Reality: Prototype beans are always created on demand -- @Lazy has no effect on them.

Mistake: @Lazy significantly reduces memory usage

Reality: @Lazy improves startup time but does NOT reduce memory -- the bean is still created eventually. Use @ConditionalOn... for truly optional beans.

Mistake: @Lazy on @Autowired prevents circular dependency issues

Reality: It can actually obscure circular dependencies -- the exception may be thrown at first use rather than startup.

Mistake: @Lazy proxies are transparent

Reality: CGLIB proxies cannot proxy final classes or final methods. If your bean class is final, @Lazy injection will fail.

I ApplicationContext Event System

ContextRefreshedEvent, custom events, @EventListener

1. Plain English Explanation

Spring's event system allows beans to communicate without knowing about each other -- classic Observer pattern. Any bean can publish an event by calling `applicationContext.publishEvent(event)`. Any bean can listen by implementing `ApplicationListener` or by annotating a method with `@EventListener`. Spring provides built-in container lifecycle events (`ContextRefreshedEvent`, `ContextClosedEvent`, etc.) and you can define custom events by extending `ApplicationEvent`.

2. Real-World Analogy

Think of a radio broadcast system. The station (publisher) broadcasts on a frequency (event type). Anyone with a tuned radio (listener) hears the message -- without the station knowing who is listening. Spring acts as the radio tower (`SimpleApplicationEventMulticaster`) routing events from publishers to matching listeners.

3. Minimal Working Code Example

```
// 1. Define a custom event
public class OrderPlacedEvent extends ApplicationEvent {
    private final String orderId;
    public OrderPlacedEvent(Object source, String orderId) {
        super(source);
        this.orderId = orderId;
    }
    public String getOrderId() { return orderId; }
}

// 2. Publish the event
@Service
public class OrderService {
    @Autowired
    private ApplicationEventPublisher publisher;

    public void placeOrder(Order order) {
        publisher.publishEvent(new OrderPlacedEvent(this, order.getId()));
    }
}

// 3a. Listen with interface
@Component
public class EmailNotifier
    implements ApplicationListener<OrderPlacedEvent> {
    @Override
    public void onApplicationEvent(OrderPlacedEvent event) {
        sendEmail("Order " + event.getOrderId() + " confirmed");
    }
}

// 3b. Listen with annotation (preferred)
@Component
public class InventoryUpdater {
    @EventListener
    public void onOrderPlaced(OrderPlacedEvent event) {
```

```

        updateInventory(event.getOrderId());
    }

    @EventListener(condition = "#event.orderId.startsWith('RUSH')")
    public void onRushOrder(OrderPlacedEvent event) {
        prioritizeShipping(event.getOrderId());
    }

    @Async
    @EventListener
    public void onOrderAsync(OrderPlacedEvent event) {
        sendSmsNotification(event.getOrderId()); // non-blocking
    }
}

```

4. Diagram -- Event System

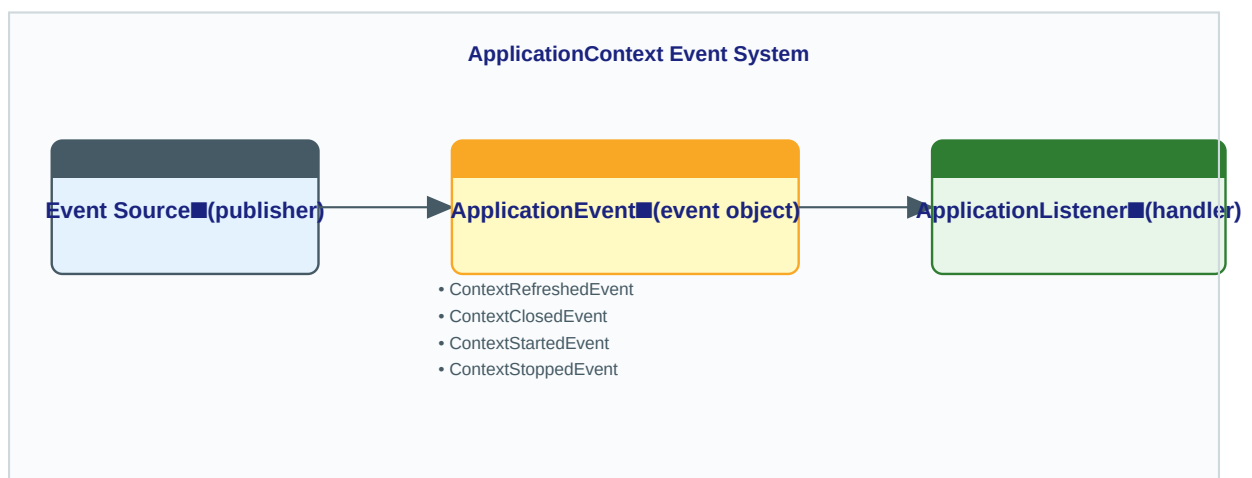


Fig 1-1: Event flows from publisher through ApplicationEventMulticaster to all matching listeners

5. Behind the Scenes -- Spring Internals

The event bus is `SimpleApplicationEventMulticaster`, initialized in `AbstractApplicationContext.initApplicationEventMulticaster()`. `publishEvent()` calls `multicastEvent(ApplicationEvent)`, which iterates all registered `ApplicationListeners` and calls `onApplicationEvent()` synchronously by default (unless a `TaskExecutor` is set). `@EventListener` methods are processed by `EventListenerMethodProcessor` (a `SmartInitializingSingleton`), which wraps each annotated method in an `ApplicationListenerMethodAdapter`. Built-in events:

```

ContextRefreshedEvent // end of refresh() -- all beans ready
ContextStartedEvent   // after ctx.start() -- Lifecycle beans started
ContextStoppedEvent   // after ctx.stop() -- Lifecycle beans stopped
ContextClosedEvent     // after ctx.close() -- all singletons destroyed

// Web-specific (SpringBoot):
ServletWebServerInitializedEvent // after embedded server starts

// Transaction-bound events (Spring 4.2+):
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onOrderCommitted(OrderPlacedEvent event) {
    // Only fires if surrounding transaction committed
    notifyExternalSystem(event.getOrderId());
}

```

For async events, set a `TaskExecutor` on `SimpleApplicationEventMulticaster`: `multicaster.setTaskExecutor(new SimpleAsyncTaskExecutor())`. For transactional events, `TransactionalEventListenerFactory` registers `ApplicationListenerMethodTransactionalAdapter`, which delegates to `TransactionSynchronizationManager` to register the listener call as an after-commit synchronization.

6. Realistic Practical Example

```
// Domain event pattern with Spring Data
@Entity
public class Order extends AbstractAggregateRoot<Order> {
    public Order confirm() {
        this.status = Status.CONFIRMED;
        registerEvent(new OrderConfirmedEvent(this));
        return this;
    }
}

@Component
public class OrderEventHandlers {

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onOrderConfirmed(OrderConfirmedEvent event) {
        // Safe: only fires after DB commit
        emailService.sendConfirmation(
            event.getOrder().getCustomerEmail());
    }

    @Async
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void indexForSearch(OrderConfirmedEvent event) {
        searchIndex.index(event.getOrder()); // background, non-blocking
    }

    @EventListener(ContextRefreshedEvent.class)
    public void onStartup() {
        cacheWarmupService.warmUp();
    }
}
```

7. Common Mistakes and Misconceptions

Mistake: Events are async by default

Reality: No. Events are synchronous and single-threaded by default. The publisher waits for all listeners. Use `@Async` to make listeners non-blocking.

Mistake: `ContextRefreshedEvent` fires once

Reality: In parent-child context setups (e.g., Spring MVC with `DispatcherServlet`), `ContextRefreshedEvent` fires for EACH context. Guard with a flag.

Mistake: `@EventListener` can return void only

Reality: Wrong -- `@EventListener` methods can return an event object, which Spring re-publishes automatically. Useful for event chaining.

Mistake: `publishEvent()` requires extending `ApplicationEvent`

Reality: Since Spring 4.2 you can publish any POJO as an event. Spring wraps it in `PayloadApplicationEvent` automatically.

Bean Lifecycle Phase Reference

Phase	What Happens	Key Class / Annotation
1. Load Definition	BeanDefinition registered from XML/@Component/@Bean	ClassPathBeanDefinitionScanner, ConfigurationClassParser
2. Instantiate	Constructor called (or CGLIB subclass for @Configuration)	AbstractAutowireCapableBeanFactory.createBeanInstance()
3. Populate Properties	@Autowired, @Value, setter injection resolved	AutowiredAnnotationBeanPostProcessor
4. BeanNameAware	setBeanName(beanName) called	AbstractAutowireCapableBeanFactory.invokeAwareMethods()
5. BeanFactoryAware	setBeanFactory(beanFactory) called	AbstractAutowireCapableBeanFactory.invokeAwareMethods()
6. ApplicationContextAware	setApplicationContext(ctx) called via BPP	ApplicationContextAwareProcessor
7. BPP Before Init	postProcessBeforeInitialization() for ALL registered BPPs	All BeanPostProcessors in order
8. @PostConstruct	JSR-250 init method; then afterPropertiesSet()	CommonAnnotationBeanPostProcessor
9. init-method	@Bean(initMethod) or XML init-method attribute	AbstractAutowireCapableBeanFactory.invokeInitMethods()
10. BPP After Init	postProcessAfterInitialization() -- AOP proxy created here	AbstractAutoProxyCreator
11. In Service	Bean is fully initialized and in the singleton cache	DefaultSingletonBeanRegistry.singletonObjects
12. @PreDestroy	JSR-250 destroy; then DisposableBean.destroy()	CommonAnnotationBPP (DestructionAware)

Extension Point Comparison

Extension	Operates On	Timing	Use Case
BeanFactoryPostProcessor	BeanDefinitions	After all defs loaded, before any instance	Modify class, scope, property of a bean definition
BDRegistryPostProcessor	BeanDefinitionRegistry	Before BFPP -- earliest possible	Register new bean definitions dynamically

BeanPostProcessor	Bean instances	Wraps init of every bean	AOP proxy, annotation processing, bean replacement
@PostConstruct	Single bean instance	After injection, before in-service	Bean-local initialization needing injected fields
InitializingBean	Single bean instance	After @PostConstruct	Alternative to @PostConstruct (Spring-specific)
ApplicationListener	Container events	When event is published	React to lifecycle events, cross-bean communication

Scope Decision Guide

Scope	Instance Count	@PreDestroy Called?	Best For
singleton	1 per ApplicationContext	Yes (on ctx.close())	Stateless services, repositories, caches
prototype	New per getBean()	NO -- manage lifecycle yourself	Stateful objects, builders, non-thread-safe tools
request	1 per HTTP request	Yes (after response committed)	Request-specific data, form models
session	1 per HTTP session	Yes (on session invalidation)	User preferences, shopping carts

2

Key Internal Classes Reference

Spring source classes to know for deep understanding

Class	Package	Role
AbstractApplicationContext	context.support	Defines refresh() -- the master startup method
DefaultListableBeanFactory	beans.factory.support	Core BF implementation; holds all BeanDefinitions
DefaultSingletonBeanRegistry	beans.factory.support	Manages singletonObjects map (3-level cache)
AbstractAutowireCapableBeanFactory	beans.factory.support	doCreateBean(), populateBean(), initializeBean()
ConfigurationClassPostProcessor	context.annotation	Processes @Configuration, @Bean, @ComponentScan
AutowiredAnnotationBeanPostProcessor	beans.factory.annotation	Resolves @Autowired and @Value
CommonAnnotationBeanPostProcessor	context.annotation	Handles @PostConstruct and @PreDestroy
AbstractAutoProxyCreator	aop.framework.autoproxy	Creates AOP proxies in postProcessAfterInit
SimpleApplicationEventMulticaster	context.event	Routes publishEvent() calls to listeners
ContextAnnotationAutowireCandidateResolver	context.annotation	Builds @Lazy CGLIB proxy for deferred injection
PostProcessorRegistrationDelegate	context.support	Orchestrates BFPP and BPP registration/invoke